Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №6
на тему

ЦИФРОВАЯ ПОДПИСЬ

БГУИР КП 6-05-0612-02 015 ПЗ

Выполнил студент гр. 353502
МАКСИМЕНКОВ
Владислав Александрович

(дата, подпись студента)

Проверил ассистент каф.Информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 8
Приложение А (обязательное) Листинг программного кода 9

ВВЕДЕНИЕ

В данной лабораторной работе рассматривается электронная цифровая подпись (ЭЦП) на основе алгоритма ГОСТ Р 34.10-2012. Цифровая подпись – это реквизит электронного документа, полученный в результате криптографического преобразования и позволяющий проверить, что документ подписан владельцем определённого ключа и не изменялся после подписания. В отличие от хеш-значения, которое может вычислить любой, подпись может сформировать только тот, кто знает закрытый ключ.

Схемы цифровой подписи являются асимметричными. Владелец формирует пару ключей: закрытый ключ хранится в секрете и используется для формирования подписи, а открытый ключ распространяется свободно и используется для её проверки. Подпись вычисляется не от самого документа, а от его хеш-значения, поэтому любое изменение документа делает подпись недействительной. Таким образом, цифровая подпись обеспечивает контроль целостности документа, подтверждение его авторства и невозможность отказа от авторства.

ГОСТ Р 34.10-2012 – стандарт Российской Федерации, описывающий процессы формирования и проверки цифровой подписи. Он пришёл на смену стандарту ГОСТ Р 34.10-2001 и принят также в качестве межгосударственного стандарта ГОСТ 34.10-2018. Алгоритм работает в группе точек эллиптической кривой над конечным простым полем. Стандарт предусматривает две длины ключа – 256 и 512 бит, а для хеширования используется функция ГОСТ Р 34.11-2012 с длиной выхода 256 или 512 бит соответственно. Стойкость алгоритма основана на сложности задачи дискретного логарифмирования в группе точек эллиптической кривой: зная точки P и $Q = dP$, вычислительно невозможно найти число d.

Целью лабораторной работы является разработка программного средства на языке C++, выполняющего генерацию ключей, формирование и проверку цифровой подписи файлов на основе алгоритма ГОСТ Р 34.10-2012.

1 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы было разработано программное средство на языке C++ для формирования и проверки цифровой подписи. Все алгоритмы реализованы без использования сторонних библиотек: арифметика больших чисел, операции с точками эллиптической кривой и хеш-функция ГОСТ Р 34.11-2012, реализованная в предыдущей лабораторной работе. Программа поддерживает генерацию ключевой пары длиной 256 или 512 бит, подпись файла закрытым ключом, проверку подписи открытым ключом и проверку реализации на контрольном примере из стандарта.

Программа разделена на несколько модулей. Модуль bigint реализует арифметику больших целых чисел, модуль ec – операции с точками эллиптической кривой и наборы параметров, модуль random – генерацию случайных чисел, модуль streebog – хеш-функцию, модуль gost3410 – генерацию ключей, формирование и проверку подписи, а главный модуль main.cpp отвечает за разбор аргументов командной строки и вывод результатов.

В модуле bigint большое число хранится как массив 32-битных слов, начиная с младшего. Сложение, вычитание и умножение выполняются «столбиком» с переносом в 64-битной переменной, а деление с остатком – по алгоритму Кнута. На их основе реализованы операции по модулю: сложение, вычитание, умножение и возведение в степень методом «возведения в квадрат и умножения». Модули p и q являются простыми числами, поэтому обратный элемент вычисляется по малой теореме Ферма:

$$a^{-1} = a^{p-2} \bmod p.$$

В стандарте используется эллиптическая кривая в форме Вейерштрасса над простым полем $\mathrm{GF}(p)$:

$$y^2 = x^3 + ax + b \bmod p.$$

Параметрами схемы являются модуль p, коэффициенты a и b, простой порядок q циклической подгруппы точек кривой и точка $P = (x_P, y_P)$ порядка q, порождающая эту подгруппу. Точки кривой образуют группу, в которой нейтральным элементом является бесконечно удалённая точка O. Сумма точек (x_1, y_1) и (x_2, y_2) вычисляется по формулам

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1,$$

где $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$ при сложении различных точек и $\lambda = \frac{3x_1^2 + a}{2y_1}$ при удвоении точки. В этих формулах на каждом шаге требуется вычисление обратного элемента, поэтому в программе точки представляются в якобиевых координатах (X, Y, Z), где $x = \frac{X}{Z^2}$, $y = \frac{Y}{Z^3}$. В таком представлении сложение и удвоение выполняются только умножениями, а обратный элемент вычисляется один раз при переводе результата обратно в обычные координаты. Произведение kP вычисляется методом «удвоения и сложения» по двоичной записи числа k.

В программе используются два набора параметров: для ключей длиной 256 бит – набор id-GostR3410-2001-CryptoPro-A-ParamSet с модулем $p = 2^{256} - 617$, для ключей длиной 512 бит – набор id-tc26-gost-3410-12-512-paramSetA с

модулем $p = 2^{512} - 569$. Кроме того, для проверки реализации используются параметры контрольного примера из приложения стандарта.

Генерация ключей выполняется функцией generateKeys. Закрытым ключом является случайное число d, $0 < d < q$, а открытым – точка кривой

$Q = dP$.

Для получения случайных чисел используется криптостойкий генератор операционной системы. Ключи сохраняются в текстовые файлы: открытый ключ содержит длину ключа и координаты точки Q, закрытый – длину ключа и число d.

Формирование подписи сообщения M выполняется функцией sign по алгоритму стандарта:

1 Вычислить хеш-значение $\overline{h} = H(M)$ по ГОСТ Р 34.11-2012 с длиной выхода, равной длине ключа.

2 Вычислить число α, двоичным представлением которого является $\overline{h}$, и определить $e = \alpha \bmod q$. Если $e = 0$, то принять $e = 1$.

3 Сгенерировать случайное число k, $0 < k < q$.

4 Вычислить точку $C = kP$ и определить $r = x_C \bmod q$. Если $r = 0$, вернуться к шагу 3.

5 Вычислить $s = (rd + ke) \bmod q$. Если $s = 0$, вернуться к шагу 3.

6 Подписью сообщения является пара чисел (r, s).

Проверка подписи выполняется функцией verify. Сначала проверяется, что $0 < r < q$ и $0 < s < q$, затем по сообщению вычисляется число e так же, как при подписи, и выполняются вычисления

$v = e^{-1} \bmod q, \quad z_1 = sv \bmod q, \quad z_2 = -rv \bmod q,$

$C = z_1 P + z_2 Q, \quad R = x_C \bmod q.$

Подпись признаётся верной, если $R = r$. Правильность проверки следует из того, что для верной подписи

$z_1 P + z_2 Q = (sv - rvd)P = v(s - rd)P = vkeP = kP.$

Если сообщение было изменено, то изменится число e, и полученная точка C не совпадёт с точкой kP, использованной при подписи.

Для запуска программы предусмотрены следующие операции:

```
gost3410 keygen [-b 256|512] public.key private.key
gost3410 sign private.key input.txt input.sig
gost3410 verify public.key input.txt input.sig
gost3410 test
```

Первая команда создаёт ключевую пару, вторая формирует подпись файла, третья проверяет подпись, а четвёртая выполняет проверку реализации. Для этого используется контрольный пример из приложения ГОСТ Р 34.10-2012: по известным параметрам кривой, закрытому ключу d, числу e и случайному числу k вычисляются открытый ключ Q и подпись (r, s), которые сравниваются с приведёнными в стандарте, после чего проверяется, что эталонная подпись принимается, а подпись с изменённым значением s отклоняется. Затем для обоих наборов параметров генерируются случайные ключи, подписывается сообщение и проверяется, что подпись верна для исходного сообщения и неверна для изменённого. Результат проверки продемонстрирован на рисунке 1.1.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab6$ ./build/gost3410 test
GOST example Q = dP  OK
GOST example sign    OK
GOST example verify  OK
GOST example bad s   OK
256 sign/verify      OK
256 changed text     OK
512 sign/verify      OK
512 changed text     OK
Все тесты пройдены
```

Рисунок 1.1 – Проверка реализации

Все вычисленные значения совпали с приведёнными в стандарте, что подтверждает правильность реализации. Генерация ключевой пары и содержимое файла открытого ключа продемонстрированы на рисунке 1.2.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab6$ ./build/gost3410 keygen examples/publ
ic.key examples/private.key
Открытый ключ: examples/public.key
Закрытый ключ: examples/private.key

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab6$ cat examples/public.key
bits: 256
Qx: f8cf92f022f54bd60175077595af4a89ea8a1b33a986f86ce7eef6d7024f0f31
Qy: 53baff725a2d68c8d6a6c590a21189af694c0778fb9766dad034312e714f884f
```

Рисунок 1.2 – Генерация ключевой пары

Далее тестовый файл input.txt был подписан закрытым ключом, что продемонстрировано на рисунке 1.3. Поскольку число k выбирается случайно, при повторном подписании того же файла получается другая подпись, которая также является верной.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab6$ ./build/gost3410 sign examples/privat
e.key examples/input.txt examples/input.sig
r: 5b0def2b2fb98d31c0100eaaac14abc9ac89c4508e8717a3fc66ed02ae20e0f7
s: f0611a870adc59817474b78697e857076aab71870f8f51acb8934fb306d9dead
Подпись сохранена: examples/input.sig
```

Рисунок 1.3 – Формирование подписи файла

Проверка подписи исходного файла открытым ключом завершилась успешно, что продемонстрировано на рисунке 1.4.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab6$ ./build/gost3410 verify examples/publ
ic.key examples/input.txt examples/input.sig
Подпись верна
```

Рисунок 1.4 – Проверка подписи исходного файла

Затем был создан файл modified.txt, в котором по сравнению с исходным изменена сумма в последней строке. Как видно на рисунке 1.5, подпись для изменённого файла признана неверной.

```
vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab6$ diff examples/input.txt examples/modi
fied.txt
10c10
< Сумма к оплате: 1000 рублей.
---
> Сумма к оплате: 9000 рублей.

vlad@vlad-HP-Pavilion-Laptop-15-eh1xxx:~/Desktop/course4/МЗИ/Lab6$ ./build/gost3410 verify examples/publ
ic.key examples/modified.txt examples/input.sig
Подпись неверна
```

Рисунок 1.5 – Проверка подписи изменённого файла

Таким образом, в ходе работы была реализована схема цифровой подписи ГОСТ Р 34.10-2012 на эллиптических кривых с длиной ключа 256 и 512 бит с поддержкой работы с файлами.

ВЫВОД

В ходе выполнения лабораторной работы был изучен алгоритм цифровой подписи ГОСТ Р 34.10-2012 и разработано программное средство на языке C++, реализующее генерацию ключевой пары, формирование и проверку цифровой подписи файлов.

В программе самостоятельно реализованы арифметика больших целых чисел, операции сложения, удвоения и умножения точек эллиптической кривой в якобиевых координатах, а также алгоритмы формирования и проверки подписи с использованием хеш-функции ГОСТ Р 34.11-2012. Корректность реализации подтверждена совпадением результатов с контрольным примером из стандарта.

Проведённое тестирование показало, что подпись исходного файла успешно проходит проверку, а изменение содержимого файла или использование другого открытого ключа приводит к тому, что подпись признаётся неверной.

В результате выполнения лабораторной работы были получены практические навыки реализации асимметричных криптографических алгоритмов на эллиптических кривых, работы с большими целыми числами и организации формирования и проверки цифровой подписи электронных документов.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – gost3410.h

```
#pragma once

#include "bigint.h"
#include "ec.h"

#include <cstdint>
#include <string>
#include <vector>

namespace gost {

struct PrivateKey {
    int bits = 256;
    BigInt d;

    void save(const std::string& path) const;
    static PrivateKey load(const std::string& path);
};

struct PublicKey {
    int bits = 256;
    Point Q;

    void save(const std::string& path) const;
    static PublicKey load(const std::string& path);
};

struct Signature {
    BigInt r, s;

    void save(const std::string& path, int bits) const;
    static Signature load(const std::string& path);
};

void generateKeys(int bits, PrivateKey& priv, PublicKey& pub);

BigInt messageToE(const std::vector<uint8_t>& msg, const Curve& c);
bool signWithK(const Curve& c, const BigInt& d, const BigInt& e,
               const BigInt& k, Signature& sig);
bool verifyE(const Curve& c, const Point& Q, const BigInt& e,
             const Signature& sig);

Signature sign(const PrivateKey& key, const std::vector<uint8_t>& msg);
bool verify(const PublicKey& key, const std::vector<uint8_t>& msg,
            const Signature& sig);

std::vector<uint8_t> readFile(const std::string& path);
void writeFile(const std::string& path, const std::vector<uint8_t>& data);

}
```

Листинг А.2 – gost3410.cpp

```
#include "gost3410.h"

#include "random.h"
```

```
#include "streebog.h"

#include <fstream>
#include <iterator>
#include <map>
#include <sstream>
#include <stdexcept>

namespace gost {

namespace {

using Fields = std::map<std::string, std::string>;

Fields readFields(const std::string& path) {
    const auto data = readFile(path);
    std::istringstream in(std::string(data.begin(), data.end()));
    Fields f;
    std::string line;
    while (std::getline(in, line)) {
        if (!line.empty() && line.back() == '\r') line.pop_back();
        if (line.empty()) continue;
        const size_t colon = line.find(": ");
        if (colon == std::string::npos)
            throw std::runtime_error(path + ": неверная строка \"" + line +
                                     "\"");
        f[line.substr(0, colon)] = line.substr(colon + 2);
    }
    return f;
}

const std::string& field(const Fields& f, const std::string& key,
                         const std::string& path) {
    const auto it = f.find(key);
    if (it == f.end())
        throw std::runtime_error(path + ": нет поля \"" + key + "\"");
    return it->second;
}

void writeText(const std::string& path, const std::string& text) {
    writeFile(path, std::vector<uint8_t>(text.begin(), text.end()));
}

}

void PrivateKey::save(const std::string& path) const {
    std::ostringstream out;
    out << "bits: " << bits << "\n";
    out << "d: " << d.toHex(bits / 4) << "\n";
    writeText(path, out.str());
}

PrivateKey PrivateKey::load(const std::string& path) {
    const Fields f = readFields(path);
    PrivateKey k;
    k.bits = std::stoi(field(f, "bits", path));
    k.d = BigInt::fromHex(field(f, "d", path));
    const Curve& c = curveFor(k.bits);
    if (k.d.isZero() || !(k.d < c.q))
        throw std::runtime_error(path + ": недопустимый закрытый ключ");
    return k;
}
```

```
void PublicKey::save(const std::string& path) const {
    std::ostringstream out;
    out << "bits: " << bits << "\n";
    out << "Qx: " << Q.x.toHex(bits / 4) << "\n";
    out << "Qy: " << Q.y.toHex(bits / 4) << "\n";
    writeText(path, out.str());
}

PublicKey PublicKey::load(const std::string& path) {
    const Fields f = readFields(path);
    PublicKey k;
    k.bits = std::stoi(field(f, "bits", path));
    k.Q = {BigInt::fromHex(field(f, "Qx", path)),
           BigInt::fromHex(field(f, "Qy", path)), false};
    if (!curveFor(k.bits).contains(k.Q))
        throw std::runtime_error(path + ": точка Q не лежит на кривой");
    return k;
}

void Signature::save(const std::string& path, int bits) const {
    std::ostringstream out;
    out << "r: " << r.toHex(bits / 4) << "\n";
    out << "s: " << s.toHex(bits / 4) << "\n";
    writeText(path, out.str());
}

Signature Signature::load(const std::string& path) {
    const Fields f = readFields(path);
    return {BigInt::fromHex(field(f, "r", path)),
            BigInt::fromHex(field(f, "s", path))};
}

void generateKeys(int bits, PrivateKey& priv, PublicKey& pub) {
    const Curve& c = curveFor(bits);
    priv.bits = pub.bits = bits;
    priv.d = randomBelow(c.q);
    pub.Q = c.multiply(priv.d, c.P);
}

BigInt messageToE(const std::vector<uint8_t>& msg, const Curve& c) {
    const BigInt alpha = BigInt::fromBytesLE(Streebog::hash(msg, c.bits));
    const BigInt e = alpha % c.q;
    return e.isZero() ? BigInt(1) : e;
}

bool signWithK(const Curve& c, const BigInt& d, const BigInt& e,
               const BigInt& k, Signature& sig) {
    const Point C = c.multiply(k, c.P);
    sig.r = C.x % c.q;
    if (sig.r.isZero()) return false;
    sig.s = addMod(mulMod(sig.r, d, c.q), mulMod(k, e, c.q), c.q);
    return !sig.s.isZero();
}

bool verifyE(const Curve& c, const Point& Q, const BigInt& e,
             const Signature& sig) {
    if (sig.r.isZero() || sig.s.isZero() || !(sig.r < c.q) ||
        !(sig.s < c.q))
        return false;
    const BigInt v = invMod(e, c.q);
    const BigInt z1 = mulMod(sig.s, v, c.q);
```

```
    const BigInt z2 = subMod(0, mulMod(sig.r, v, c.q), c.q);
    const Point C = c.add(c.multiply(z1, c.P), c.multiply(z2, Q));
    if (C.infinity) return false;
    return C.x % c.q == sig.r;
}

Signature sign(const PrivateKey& key, const std::vector<uint8_t>& msg) {
    const Curve& c = curveFor(key.bits);
    const BigInt e = messageToE(msg, c);
    Signature sig;
    while (!signWithK(c, key.d, e, randomBelow(c.q), sig)) {
    }
    return sig;
}

bool verify(const PublicKey& key, const std::vector<uint8_t>& msg,
            const Signature& sig) {
    const Curve& c = curveFor(key.bits);
    return verifyE(c, key.Q, messageToE(msg, c), sig);
}

std::vector<uint8_t> readFile(const std::string& path) {
    std::ifstream in(path, std::ios::binary);
    if (!in) throw std::runtime_error("не удаётся открыть файл: " + path);
    return {std::istreambuf_iterator<char>(in),
            std::istreambuf_iterator<char>()};
}

void writeFile(const std::string& path, const std::vector<uint8_t>& data) {
    std::ofstream out(path, std::ios::binary);
    if (!out) throw std::runtime_error("не удаётся создать файл: " + path);
    out.write(reinterpret_cast<const char*>(data.data()),
              static_cast<std::streamsize>(data.size()));
    if (!out) throw std::runtime_error("ошибка записи в файл: " + path);
}

}
```